PTAssistant

An AI Personal Training Assistant

2025-2026 Spring Semester

SE 380: Mobile Application Development

Term Project Report

Sinan Mert Sener - 20190602037

Instructor: Gazihan Alankus

Video: [PASTE YOUR YOUTUBE URL HERE]

1. Introduction

PTAssistant is a mobile app that works like a personal trainer you carry in your pocket. Instead of writing my workouts on random notes and guessing what to do next, I wanted one place that knows my body, my goals and my injuries, and that can build and change my training for me just by talking to it.

The idea came from my own time at the gym. I was tracking workouts on scattered notes and could never really follow my progress on each exercise. I also wanted something that could safely adjust my training the moment I got injured or started lifting too heavy, instead of paying for a personal trainer that most people cannot afford. PTAssistant is my answer to that.

The app is written in Flutter and runs on Android. All the user data lives in Firebase (authentication, the Cloud Firestore database and storage). The AI coach runs on Google Gemini through the Firebase AI package, and it can actually change things in the app (your profile, your programs, your notes and your reminders) by calling tools, not just chat back. The whole app works in English and Turkish.

2. Project Overview and Features

2.1 AI Coach (the chat)

The main feature is the AI Coach. It is a chat where you talk to a trainer that already knows your profile. You can ask it to build a program, change one, check your form, plan your week or work around an injury, in plain English or Turkish. The answer streams in word by word like ChatGPT.

What makes it more than a normal chatbot is that it can actually do things in the app through 14 tools: read and update your profile, list, create and edit programs and set the active one, read, create and edit notes, schedule a reminder, and look something up on the web with Google Search when it needs a real source. When the coach builds a program inside the chat, an "Add to My Programs" button appears under that message so you can save it with one tap.

2.2 Training programs

A program has a title and a list of days, and each day has exercises with sets, reps, rest time and notes. Programs are marked as either AI or manual depending on how they were made, and only one program is active at a time. You can build a program by hand in the editor, or let the coach make one and import it from the chat. You can edit, duplicate, set active or delete any program.

2.3 Guided workout sessions and streak

When it is time to train, you start a session from your program. The session screen shows each exercise with one row per set, where you type the weight (kg) and the reps you actually did. When you finish, the app saves a workout log and adds up your total volume (reps times weight). Finishing sessions builds a day streak that shows on the home screen.

2.4 Onboarding and profile

The first time you sign in you go through a 3 step onboarding that builds the profile the AI reasons over: step 1 is who you are (name, date of birth, sex), step 2 is body metrics (height, weight, experience level, weekly sessions), and step 3 is goals, equipment and injuries. Later you can change all of this on the Profile screen, set a profile photo (the app lets you pick and crop an image), and edit your goals, equipment and injuries.

2.5 Notes

There is a simple notes feature for training thoughts. Notes support markdown, tags, pinning and search, and they save automatically as you type. The AI coach can also read and write your notes through its tools.

2.6 Notifications and the daily AI tip

The home screen shows a short daily tip that the AI writes for you based on your profile and program, with Accept and Dismiss buttons. The app also schedules local notifications: reminders that the coach can set for you, and automatic "come back" nudges that fire after a few days of not training so you do not lose your streak. Firebase Cloud Messaging is also wired in and the device token is stored per user.

2.7 Safety guardrails

Because this is a fitness app and not a doctor, I added safety on top of the AI. A separate small Gemini model checks every message first and decides if it is on topic, so the coach stays about training and does not get pulled into off topic or jailbreak requests. The system prompt also tells the coach not to diagnose injuries or give medical, legal or financial advice, and when injury or pain is mentioned the app adds a reminder to see a doctor.

2.8 Localization and theming

The whole app works in English and Turkish, including dates and times that follow the chosen language. There is a light and a dark theme, and you can switch the language and the theme from settings.

3. Architecture and Technologies

The app uses a feature first structure. The UI screens live under lib/features (auth, home, chat, programs, notes, profile) and the shared code lives under lib/core (models, repositories, AI, notifications, theme, router).

State management and dependency wiring use Riverpod. All Firebase access goes through small repository classes (one per data type: profile, programs, notes, chat, reminders, workouts), so the screens never talk to Firestore directly. Navigation uses go_router with a redirect guard that sends you to sign in if you are logged out, and to onboarding if your profile is not finished yet.

The backend is Firebase:

- Firebase Authentication (email and password, with password reset)
- Cloud Firestore for all data, stored under one document tree per user
- Firebase Storage for the profile photo
- Firebase Cloud Messaging for push tokens, and Firebase App Check
- Firebase AI to talk to Google Gemini (model gemini-2.5-flash-lite)

The AI coach is built on Gemini function calling. The app gives the model 14 tool definitions, runs the tools against the user's own data, feeds the results back, and loops until the model has a final answer, all while streaming the text to the screen. A second Gemini model with temperature 0 is used only as an on topic and safety classifier.

Data is private by design. In Firestore every user can only read and write their own subtree (users/{uid}/...), enforced by security rules that check the signed in user id, and everything else is denied.

The models are immutable and generated with freezed and json_serializable. The code passes flutter analyze with no issues, and there are 39 unit and widget tests covering the repositories, the guardrails, the tool registry and the streak logic, and they all pass.

4. Work Distribution

This is a solo project. I, Sinan Mert Sener, designed and built all of it: the onboarding and authentication, the AI coach with its tools and guardrails, the program editor and the session runner, the home screen, the notes, the profile, the settings, the notifications, the Firebase setup and the security rules, and the English and Turkish localization.

I used AI coding assistants while building it (see the External Factors section). I made the design decisions, chose the structure, and put the whole app together myself.

5. Improvements Since the Presentation

In the presentation I showed the app working and listed what I still wanted to finish. Compared to that plan, here is what I added before this submission:

- Streaming chat: the coach replies now stream in word by word instead of showing up all at once after a wait. This was on my "what's next" list in the presentation.
- Notifications: I added the local notification system and the automatic "come back" nudges that remind you to train after a few days away, plus reminders the coach can schedule. This was my planned "smart notifications".
- Locale aware date and time: dates and times now follow the selected language. This was on my "before submission" list.
- Full language support: I finished the English and Turkish translations across every screen and string. This was the other item on my "before submission" list.

So I finished both items I promised for the final version (full localization and locale aware dates), and on top of that I also delivered two items that were on my longer roadmap (streaming chat and notifications). The two things still on the roadmap for later, which I did not promise for this submission, are a weekly progress bar on the home screen and social or public profiles.

6. User Manual

6.1 Creating an account and onboarding

Open the app and tap Sign Up. Enter any email and a password of at least 6 characters and tap Sign Up. If you already have an account, use Sign In, and there is a Forgot password option that emails you a reset link. The first time, you go through 3 short steps: your name, date of birth and sex; then height, weight, experience level and how many sessions you train per week; then your goals, your equipment and any injuries. Tap Finish and you land on the home screen.

6.2 Home screen

The home screen greets you and shows your day streak at the top. The big card shows today's session from your active program (the day name, how many exercises and a time estimate) with a Start Session button. If you have no active program yet, it invites you to ask the AI. Below that is the daily AI tip card with Accept (which opens the coach) and Dismiss.

6.3 Talking to the AI Coach

Open the AI Coach tab. Type what you want in the box at the bottom, for example "build me a 4 day upper lower program with dumbbells" or "my shoulder hurts, adjust today", in English or Turkish, and send it. The reply streams in. When the coach uses a tool you see a small card (for example "Program Created" or "Profile Updated"). When the coach writes a full program in the chat, tap Add to My Programs under that message to save it, and a message pops up with a link to open it in Programs. To start over, tap the trash icon in the top right and confirm to clear the conversation.

6.4 Programs

The Programs tab lists your programs with a search box and filter pills (All, Active, AI, Manual). The active program has a green ACTIVE badge, and you can tap the bolt icon on any other program to make it active. Tap a program to open it. To make one by hand, tap Create New, type a title and you go straight into the editor. In the editor you add days, rename them, and add exercises; each exercise has a name, sets, reps, rest in seconds and optional notes. Tap Save when you are done. On a program's detail screen you can edit it, set it active, duplicate it, delete it, switch between days, and start a session.

6.5 Running a workout session

From a program tap Start Session. For each exercise, type the weight in kg and the reps you did for each set, and use Add Set if you need an extra set. When you are done tap Finish. The app saves the session as a workout log and it counts toward your streak.

6.6 Notes

The Notes tab lists your notes with a search box. Tap New Note to make one. A note has a title and a body that supports markdown, you can add tags, pin important notes to the top, and switch to a preview to see the formatted text. Notes save by themselves as you type. The trash icon removes a note.

6.7 Profile and settings

The Profile tab shows your photo, name and experience level, your metrics (height, weight, weekly sessions), your goals and your injuries, and you can edit each of them there. Tap the camera icon to pick and crop a profile photo. The settings icon in the top right opens Settings, where you can switch the language (English or Turkish), switch the theme (System, Light or Dark), turn notifications on or off, sign out, or delete your account (this asks you to confirm because it cannot be undone).

7. External Factors

I want to be clear about the outside help and tools I used:

- AI coding assistants: I used AI assistant tools (such as Claude) while writing, refactoring and debugging the code. I designed the app, made the decisions and assembled it myself, but AI helped me write parts of it.
- Google Gemini through Firebase AI: the AI coach and the daily tip are powered by Google's Gemini model, and the source lookup tool uses Google Search grounding.
- Firebase: Authentication, Cloud Firestore, Storage, Cloud Messaging and App Check are Google's backend services.
- Open source packages from pub.dev: the main ones are flutter_riverpod, riverpod_annotation, go_router, the firebase packages (core, auth, cloud_firestore, storage, messaging, app_check), firebase_ai, flutter_local_notifications, timezone, freezed, json_serializable, image_picker, image_cropper, flutter_markdown, shared_preferences, intl, collection and rxdart.
- Documentation: I used the official Flutter and Firebase documentation while building.

This project was made only for SE 380. It was not a joint project with any other class, and it was not reused from any other course.